

Chastity's LaTeX Book

Chastity White Rose

October 3, 2026

Contents

Introduction	3
1 What is LaTeX?	5
2 Why am I Learning LaTeX?	7
3 Goals of this LaTeX Book	9
4 Including Source Code	11
5 Including Images	13
6 Changing Font Size	17

Introduction

This short example book was written in LaTeX by Chastity to remind herself why she was learning LaTeX. It was converted to PDF by the pdflatex program running on Debian Linux.

LaTeX is a plain text language for describing books meant to be read in paperback form or as digital PDF files.

Chapter 1

What is LaTeX?

LaTeX is a language for describing documents that you might want to print or just view on a computer. It is similar to what HTML is for web pages because you have tags representing different kinds of elements. Many people say it is extremely helpful for making scientific papers or books representing math formulas that are hard to represent using traditional word processors or Markdown.

Because it is a plain text format, it can be edited using any text editor and doesn't require using special software. In particular, it avoids the need for Microsoft Word that many people are told to use. LibreOffice is a good GUI alternative to Microsoft Word but sometimes plain text is the fastest, especially for editing a document using a terminal only environment on a Linux that may not even have the X Window System installed.

Chapter 2

Why am I Learning LaTeX?

My own interest in LaTeX began when Matthew Moodie was editing my DOS edition of Assembly Arithmetic Algorithms. He told me he was using LaTeX in combination with Markdown and converting with Pandoc. I have written many books using Markdown and my files usually make great ebooks but when it comes to paperback books, my knowledge and skill is more limited and I rely on LibreOffice or Leanpub for converting my books into readable PDF files.

By using other people or software, I am trusting them to make the decisions on things like paper size or font size. However, this is not good enough for an autistic person like me who wants to control everything about my computer usage, including the programs I write and books I write about those programs!

By learning more about LaTeX, I hope to fill this gap in my knowledge so that I can use it to write a future book using it and not rely on other people or GUI tools that I don't understand well. Plain text has always been my style of getting work done on the computer. LaTeX is just as capable as Markdown but perhaps more expressive in formatting.

Chapter 3

Goals of this LaTeX Book

The books I usually write are very simple. At least, a book contains text separated into chapters and sections within chapters. Sometimes this text is bold or italic, and sometimes, the text is meant to be separated into a code block showing the source code of a program.

More complex books require the use of images, such as my book: Chastity's Chess Chapters. Images are the best way to render anything where the look must be perfect, but they also take time to create and are tricky to include in books because GUI editors like to choose where they appear and force the user to click and drag with a mouse to reposition them or right click and open a dialog box to resize them.

The headache I just mentioned about images is one I experienced and ultimately what forced me to learn Markdown. The goals of this book are to learn how to use LaTeX to do everything I could have done with Markdown and perhaps more as I learn the capabilities of LaTeX and how I can use it to make books on Arithmetic and Computer Programming Languages.

Chapter 4

Including Source Code

Because programming books are my favorite thing to write, it is essential to be able to include code listings which will retain their spacing and not be interpreted as paragraphs of text.

The following listing is a C function I wrote which prints any zero terminated string of bytes. It is one of the functions I am most proud of because the algorithm used is the same as the one used for the `putstring` function in all of my editions of *Assembly Arithmetic Algorithms*

```
int putstring(const char *s)
{
    int count=0;           /*used to count how many bytes will be written*/
    const char *p=s;       /*pointer used to find terminating zero of string*/
    while(*p){p++;}        /*loop until zero found and immediately exit*/
    count=p-s;             /*count is the difference of pointers p and s*/
    fwrite(s,1,count,stdout); /*https://cppreference.com/w/c/io/fwrite.html*/
    return count;          /*return how many bytes were written*/
}
```

A slightly more complicated function and its global variables are shown below. The `getstring` function returns a zero terminated string that can be input by the user. I used this function in my `chastdin` calculator. This means `getstring` is the opposite of `putstring`, as its name would imply.

```
/*variable to store how many characters were read into the string last time*/
int read_count=0;
char last_char=0;
```

```
/*
this is my basic function for recieving input from the keyboard
it uses a global variable to store the string into
However, it returns a pointer to this string which makes it really convenient if I need
to read something and then immediately print it or copy it to another variable
```

```
It reads characters into the string and stops at the first whitespace character.
This can be a regular space, tab, or newline.
```

```
My reasoning for this is because I plan for multiple strings to exist on one line.
```

```
This is similar to how a command and its arguments are entered on the command line.
```

```
By stopping every time a separating whitespace character is found, the tokens can be processed manually
*/
```

```
char *getstring()
{
    char *s=main_string,*p=s,c=0;
```

```
while( c!=' ' && c!='\n' && c!='\t')
{
    read_count=fread(p,1,1,stdin); /*read exactly 1 byte*/
    if(read_count==0) /*if failed to read one byte, print EOF*/
    {
        *p=0; /*terminate this string with zero*/
        return s; /*return the null string because there is nothing left to read*/
    }
    c=*p; /*character equals the last character read*/
    p++; /*increment the pointer to the next byte*/
}
last_char=c;
*--p=0; /*go back to last character (space,newline,tab) and replace with zero*/
read_count=p-s;
return s;
}
```

Chapter 5

Including Images

Images should usually be used quite sparingly in books designed to be printed as paperbacks, however they are essential for showing geometric shapes in a way text never can. The following images in this chapter were included in my book: *Chastity's Code Cookbook*. I have included them here mostly as a demonstration that I figured out how to include images in a LaTeX document.

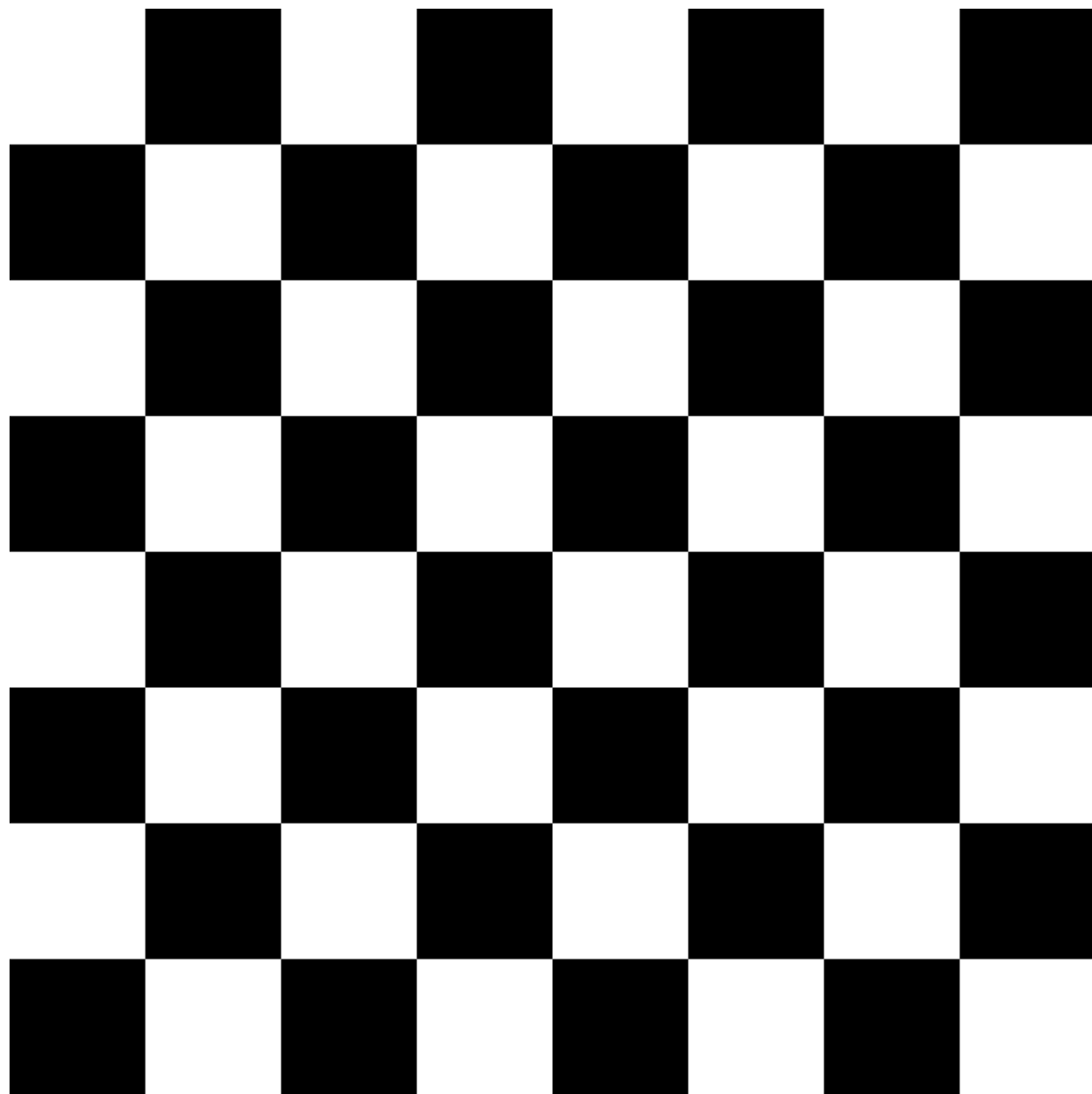


Figure 5.1: A checker board

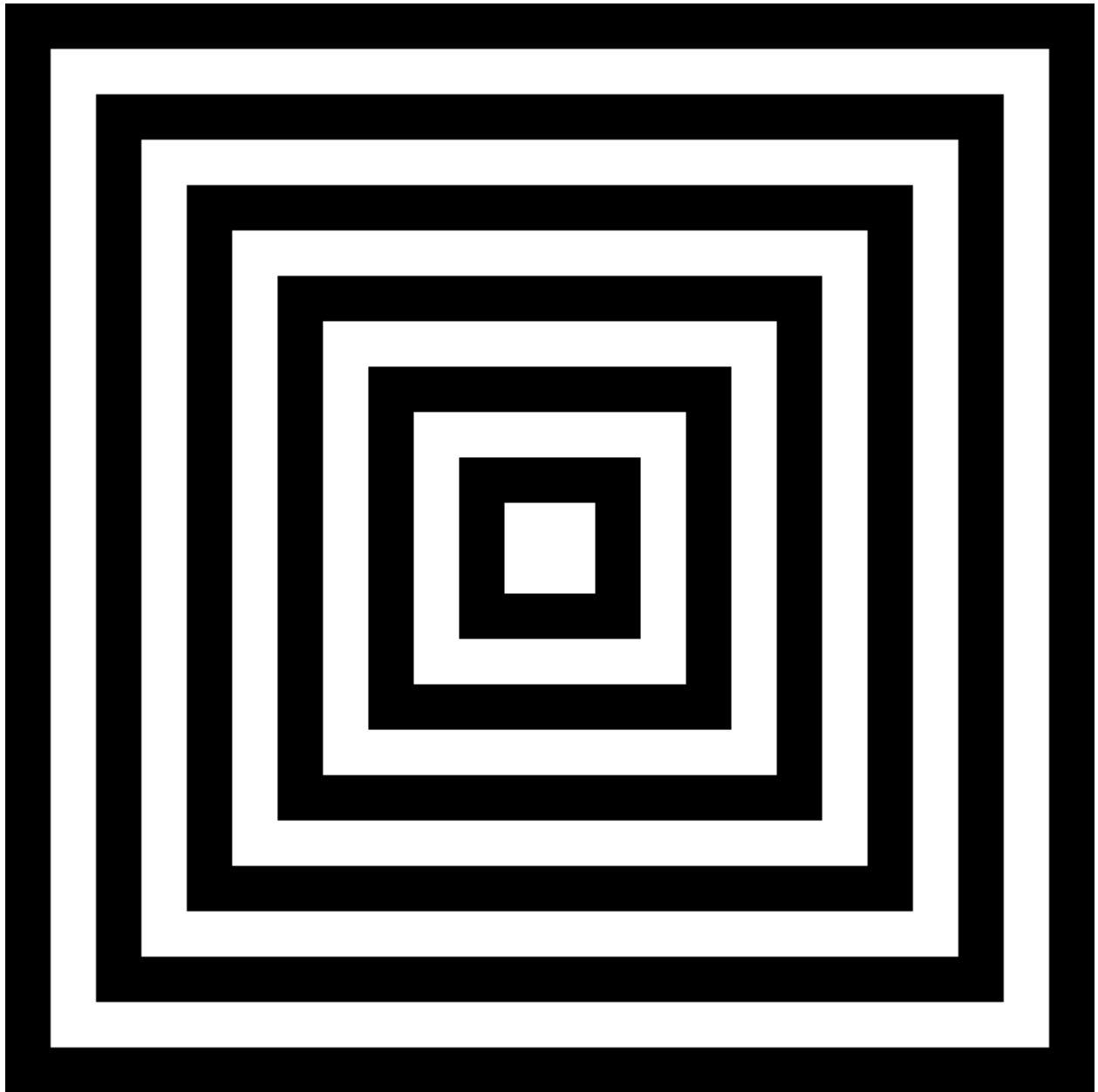


Figure 5.2: A square target

Chapter 6

Changing Font Size

Most of the time, the default font size for LaTeX doesn't matter very much. After all, most PDF files are read on computers and can be resized on the screen. But when printing real paperback books, there may be reasons to make text bigger.

For this purpose, there are LaTeX commands for common sizes. They can be used to temporarily make certain sections a different size of text. Here is a code sample and the output it generates.

```
\tiny tiny
\scriptsize scriptsize
\footnotesize footnotessize
\small small
\normalsize normalsize
\large large
\Large Large
\LARGE LARGE
\huge huge
\Huge Huge
```

tiny scriptsize footnotessize small normalsize large Large LARGE huge Huge

There are some references I use to remind myself how to change font sizes in a document. It can either be changed individually with the above keywords or for an entire document as part of its class options.

Document-class-options
Font-sizes